



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Méréstechnika és Információs Rendszerek Tanszék

Mérnöki modellek szegmentálása

BSC ÖNÁLLÓ LABORATÓRIUM

Készítette
Lénárt Csaba Tamás

Konzulens
Dr. Salvi Péter

2026. május 29.

Tartalomjegyzék

1. Bevezetés	1
1.1. Összefoglaló	1
1.2. Motiváció	1
2. Szakirodalmi áttekintés	2
3. Implementáció	4
3.1. A projekt felépítése	4
3.2. Hálófeldolgozás és adatmodell	4
3.3. Szegmentációs algoritmus	5
3.3.1. Random walk megoldás	5
3.3.2. Automatikus seedválasztás	6
3.3.3. Régióösszevonás	7
3.4. Megjelenítés és felhasználói interakció	7
3.4.1. Renderelés bgfx-szel	7
3.4.2. Felhasználói felület	8
3.5. Implementációs különbségek	8
4. Eredmények	10
5. Összefoglalás	13
5.1. Jövőbeli tervek	13
6. Hivatkozások	14

1. fejezet

Bevezetés

1.1. Összefoglaló

A projekt célja egy olyan alkalmazás vizsgálata és továbbfejlesztése, amely mérnöki és grafikus háromdimenziós modellek szegmentálását támogatja. A feldolgozott szakirodalmi módszer a képszegmentálásban alkalmazott random walk eljárást terjeszti ki térbeli modellekre: a modell elemei közötti kapcsolatokat gráfként kezeli, és ez alapján interaktív, illetve automatikus szegmentálást tesz lehetővé. A módszer előnye, hogy a modell lapjainak és a keresett régióknak a számával jól skálázódik, zajos bemeneten is robusztus, és az eredményei további feldolgozási feladatokban, például visszamodellezésben vagy példaalapú modellezésben is felhasználhatók.

A SegMesh projekt ezt a megközelítést egy interaktív alkalmazás formájában valósítja meg. A program a modell betöltése, megjelenítése és szegmentálása mellett manuális, durva automatikus és finom automatikus munkafolyamatokat is biztosít. A megjelenítési réteg a **bgfx** könyvtárra épül, amely platformfüggetlen, grafikus API-tól független renderelő könyvtárként illeszthető saját alkalmazás- vagy motorkeretbe. Ez lehetővé teszi, hogy a projektben a szegmentálási algoritmusok és a megjelenítés viszonylag jól elkülönüljenek, miközben az eredmények közvetlenül vizsgálhatók a felhasználói felületen [7].

1.2. Motiváció

A témaválasztás kiindulópontja az volt, hogy egy háromdimenziós grafikai projektet szerettem volna készíteni, eredetileg Vulkan használatával. Egy teljes játékmotor megvalósítása azonban a fél éves keretekhez képest túl nagy és túl sokoldalú feladatnak tűnt, ráadásul nem is ez volt a cél: inkább egy jól körülhatárolható problémán keresztül szerettem volna megismerni a 3D-s megjelenítés és a hozzá kapcsolódó adatfeldolgozás alapvető kérdéseit.

A feldolgozott *Rapid and effective segmentation of 3D models using random walks* című, random walk alapú szegmentálási cikk ehhez megfelelő témát adott [1]. A háromdimenziós modellek szegmentálása elég konkrét ahhoz, hogy a projekt fókuszált maradjon, ugyanakkor tartalmaz geometriai feldolgozást, interaktív megjelenítést és felhasználói vezérlést is. A Vulkan közvetlen használata önmagában jelentős fejlesztési ráfordítást igényelt volna, ami egyetlen fél évbe nehezen fért volna bele. Ezt a problémát a **bgfx** könyvtár oldotta fel: olyan renderelési alapot biztosít, amely mellett a grafikus programozás több fontos koncepciója továbbra is megjelenik, de a fejlesztési folyamat egyszerűbb és gyorsabb marad.

2. fejezet

Szakirodalmi áttekintés

A háromdimenziós modellek szegmentálása olyan feldolgozási lépés, amely egy hálót vagy pontfelhőt egymástól elkülönülő, geometriailag vagy szemantikailag értelmezhető részekre bont. A feladat jelentősége abból adódik, hogy a nyers háromdimenziós adatok önmagukban gyakran csak pontok, élek és lapok halmazaként állnak rendelkezésre, miközben a további feldolgozás rendszerint magasabb szintű egységeket igényel. Ilyen további feladat lehet a visszamodellezés, az alakfelismerés, a modellalapú keresés, a tömörítés, a példaalapú modellezés vagy a parametrikus CAD-modell rekonstrukciója.

Theologou és munkatársai 2015-ös áttekintése alapján a hálóssegmentálási módszerek több szempont szerint csoportosíthatók. Fontos különbség, hogy a szegmentálás felülettípusokat keres-e, vagy a modell természetes alkatrészeit próbálja elválasztani. Az előbbi irány különösen a mérnöki modelleknél fontos, ahol síkok, hengerek, gömbök, átmeneti felületek és egyéb szabályos elemek azonosítása a cél. Az utóbbi inkább grafikus vagy organikus modelleknél jelenik meg, ahol a modell szemantikailag értelmezhető részei, például végtagok vagy objektumrészek érdekesek. A módszerek emellett lehetnek régióalapúak vagy határalapúak, automatikusak vagy interaktívak, illetve dolgozhatnak hierarchikus vagy egyszerű, egyszintű felosztással [2].

A mérnöki visszamodellezés irodalmában a szegmentálás szerepe különösen erősen kapcsolódik a felületillesztéshez. Váradý és munkatársai a szabályos mérnöki objektumok rekonstrukcióját olyan folyamatként tárgyalják, amelyben a mért adatokból először értelmes felületrégiókat kell elkülöníteni, majd ezekre kell analitikus felületeket illeszteni. A későbbi, 2007-es munkájukban a cél már kifejezetten a felületi struktúrák automatikus kinyerése: a modell elsődleges felületeit, az összekötő átmeneti felületeket és a csúcskörnyéki átmeneteket kell úgy azonosítani, hogy az eredmény CAD/CAM/CAE rendszerekben is használható legyen. Ez a megközelítés jól mutatja, hogy a szegmentálás minősége közvetlenül befolyásolja a későbbi rekonstrukció pontosságát és használhatóságát [3, 4].

A hálóalapú módszerek mellett a pontfelhők közvetlen szegmentálása is fontos kutatási irány. Fayolle és Pasko munkája olyan eljárást mutat be, amely rendezetlen pontfelhőn dolgozik, tehát nem feltételez háromszöghálót vagy explicit szomszédsági kapcsolatokat. A módszer előre megadott sablonprimitívekhez illeszti a pontokat, majd az illeszkedő részeket fokozatosan leválasztja. Ez a szemlélet különösen akkor hasznos, ha a bemenet közvetlenül háromdimenziós szkennelésből származik, és a háló előállítása vagy javítása külön hibaforrás lenne. A pontfelhőalapú megközelítések ugyanakkor általában nagyobb hangsúlyt helyeznek a zaj, a hiányos mintavételezés és a kiugró pontok kezelésére [5].

A parametrikus rekonstrukció felől nézve a szegmentálás a CSG-modellek visszaállításában is alapvető előfeldolgozási lépés. A Shapr3D 2020-as technikai jelentése a háromdimenziós szkennelésből kiinduló CSG-rekonstrukciót vizsgálja. Ebben a cél nem pusztán egy háromszögháló előállítása, hanem egy olyan szerkesztési történet vagy primitív-

fa megtalálása, amely gömbökből, hengerekből, téglatestekből és Boole-műveletekből építi fel a modellt. Ehhez a bemeneti adatokban fel kell ismerni azokat a geometriai részeket, amelyek egy-egy primitívhez vagy konstrukciós lépéshez tartozhatnak. Emiatt a szegmentálás nem önálló végcél, hanem egy magasabb szintű modellértelmezési folyamat része [6].

A jelen projekt szempontjából a legfontosabb kiindulópont Lai és munkatársai 2009-es, *Rapid and effective segmentation of 3D models using random walks* című cikke, amely a képszegmentálásban alkalmazott random walk módszert terjeszti ki háromdimenziós modellekre. A módszer a hálót gráfként kezeli: a lapok vagy pontok a gráf csúcsai, a köztük lévő kapcsolatok pedig élekként jelennek meg. Az interaktív változatban a felhasználó néhány seed régiót jelöl ki, majd az algoritmus azt számítja ki, hogy a gráfon véletlen sétát indítva egy adott elem melyik seedhez jutna el legnagyobb valószínűséggel. Lai és munkatársai automatikus változatot is adnak, amely először durvább túlszegmentálást hoz létre, majd a hasonló régiókat összevonja [1].

Ez az irány azért illeszkedik jól a SegMesh projekthez, mert egyszerre ad használható interaktív munkafolyamatot és automatikus szegmentálási alapot. A random walk megközelítés előnye, hogy a számítás ritka lineáris egyenletrendszer megoldására vezethető vissza, ezért közepes méretű modelleken is hatékonyan futtatható. Emellett a módszer nem kizárólag mérnöki vagy grafikus modellekre korlátozódik: Lai és munkatársai háromszöghálókön és pontfelhőkön is bemutatják, és a zajjal szembeni robusztusságát is kiemelik. A SegMesh ennek megfelelően nem teljes CAD-rekonstrukciós rendszerként működik, hanem a szegmentálási lépésre és annak interaktív vizsgálatára koncentrál.

3. fejezet

Implementáció

3.1. A projekt felépítése

A projekt felépítése néhány jól elkülöníthető modulra bontható. A cél az volt, hogy a háló betöltése, a szegmentálási algoritmus, a renderelés és a felhasználói felület külön felelősségi kör maradjon. Így a random walk algoritmus nem közvetlenül a grafikus API adataival dolgozik, hanem egy CPU oldalon előkészített hálóreprezentáción, amelyet a renderelő csak megjelenítésre használ fel.

- A `mesh_types.*` fájlok adják a közös adattípusokat. A `CpuMesh` tartalmazza a szegmentáláshoz szükséges geometriai és topológiai adatokat, a `GpuMesh` pedig a `bgfx` oldali bufferhandle-öket. A `SegmentationModelType` választja szét a grafikus és a mérnöki modellhez használt súlyozást.
- Az `obj_loader.*` felel az `.obj` fájlok betöltéséért és az `OpenMesh` által beolvasott modell átalakításáért. Ez a modul állítja elő a lapközéppontokat, normálisokat, lapterületeket, lapszomszédságot és a görbületi becsléseket.
- A `segmentation.*` tartalmazza az algoritmikus magot: a random walk megoldót, a durva és finom automatikus seedválasztást, a régiók összevonását és a kis szegmensek utólagos tisztítását.
- A `renderer.*` fájlokban található a `Renderer` osztály, amely inicializálja a `bgfx`-et, feltölti a GPU buffereket, majd kirajzolja az alapmodellt, a szegmensszíneket, a határvonalakat és a seed háromszögeket.
- A `ui.*` fájlok az `ImGui` alapú vezérlőpaneleket valósítják meg. A `RendererUiState` tárolja a kamera, a világítás, a szegmentációs mód és az automatikus szegmentálás paramétereit, a `RendererUiActions` pedig a felhasználói műveleteket adja vissza a fő ciklusnak.
- A `main.cpp` köti össze a fenti modulokat. Ez kezeli az alkalmazás fő ciklusát, a modellválasztást, a kameramozgatást, az egérrel történő háromszögválasztást, a seedek állapotát, valamint a szegmentációs előnézet újraszámítását.

3.2. Hálófeldolgozás és adatmodell

A program bemenete háromszögháló, amelyet az `OpenMesh` olvas be `.obj` fájlból. A betöltés után a modell koordinátái normalizálásra kerülnek, hogy a különböző méretű modellek hasonló kamerabeállításokkal és numerikus skálán legyenek kezelhetők. A betöltő

a modell befoglaló doboza alapján az origó közelébe tolja a geometriát, majd úgy skálazza, hogy a leghosszabb kiterjedése nagyjából két egység legyen.

A szegmentálás nem vertex-, hanem lapalapú. Minden háromszöglaphoz eltárolódik a lap középpontja, normálisa és területe. Ezeket az értékeket a **CpuMesh** tárolja. Ugyanez az adatszerkezet tartalmazza a lapok szomszédságát is: két lap akkor szomszédos, ha közös élük van. A **FaceAdjacency** ezért laponként legfeljebb három szomszédot, három élhosszt és három konkávitási skálát tartalmaz.

A renderelés igényei miatt a betöltő a háromszögek csúcsait laponként duplikálja. Ez egyszerűsíti a kirajzolást, mert minden háromszög saját renderelési indexeket kap, és a szegmenshatárok, valamint a seed háromszögek külön indexbufferekbe rendezhetők. Az algoritmus szempontjából azonban a fontos adat nem a duplikált render-vertex, hanem a lapgráf. Ebben a gráfban a csúcsok a háromszöglapok, az élek pedig a közös éllel rendelkező lapszomszédságok. A későbbi random walk szegmentálás ezen a gráfon fut.

3.3. Szegmentációs algoritmus

3.3.1. Random walk megoldás

A random walk szegmentálás alapötlete, hogy a felhasználó vagy az automatikus eljárás kijelöl néhány seed lapot, majd a program minden más lapra kiszámolja, melyik seed érhető el a legnagyobb valószínűséggel egy véletlen sétával. A véletlen séta átmeneti valószínűségeit a szomszédos lapok közötti geometriai különbségek határozzák meg: erős határ esetén kicsi, sima átmenet esetén nagy az átlépés valószínűsége.

A grafikus modellhez használt különbségfüggvény főként a szomszédos lapok normálisainak eltérésére épül. Ha f_i és f_j szomszédos lapok, normálisaik $\mathbf{N}_i$ és $\mathbf{N}_j$, akkor a grafikus mód normalizált különbsége:

$$d_{\text{graph}}(f_i, f_j) = \frac{\eta_{ij}(1 - \mathbf{N}_i \cdot \mathbf{N}_j)}{\bar{d}_{\text{graph}}}. \quad (3.1)$$

Itt η_{ij} a konkáv-konvex élek eltérő kezelésére szolgáló skálázó tényező. A képlet Lai és munkatársai grafikus modellekre használt normáliskülönbség-alapú súlyozásának felel meg, azzal a projektbeli egyszerűsítéssel, hogy a konkáv és konvex élek kezelése egy közvetlen skálázó tényezővel történik. A konkáv élek nagyobb súlyt kapnak, a konvex élek kisebbet, mert grafikus modelleknél a szemantikailag természetes határok gyakran konkáv átmeneteknél jelennek meg.

A mérnöki mód más célt követ: itt nem csak a vizuálisan természetes alkatrészek, hanem a különböző felülettípusok elválasztása is fontos. Ezért a normáliskülönbség mellett a becsült Gauss- és középgörbületi értékek változása is szerepel:

$$d_{\text{eng}}(f_i, f_j) = \max \left(\frac{1 - \mathbf{N}_i \cdot \mathbf{N}_j}{\bar{d}_N}, \frac{|K_i - K_j|}{\bar{d}_K}, \frac{|H_i - H_j|}{\bar{d}_H} \right). \quad (3.2)$$

A képletben K_i és H_i rendre az f_i laphoz tartozó becsült Gauss- és középgörbületet jelöli, a nevezők pedig a teljes modellen számolt átlagos különbségek. Ez a képlet Lai és munkatársai mérnöki modellekhez használt gondolatát követi: a felületek szétválasztásánál nemcsak a normális, hanem a görbületi viselkedés is számít. A projektben ez nem teljesen azonos az eredeti módszerrel, mert a görbületek becslése egyszerűbb. A kódban van egy külön eset is: ha két szomszédos lap geometriailag közel koplanáris, akkor a program a görbületi becslések zajosságá miatt csak a normáliskülönbségre támaszkodik.

A különbségértékből élsúly lesz. A `segmentMeshRandomWalk()` függvény a közös él hosszát is figyelembe veszi, ezért egy i és j lap közötti súly:

$$w_{ij} = |e_{ij}| \exp(-d_{ij}), \quad (3.3)$$

ahol d_{ij} a választott modell szerinti normalizált különbség. Ez a forma megegyezik Lai és munkatársai élsúlyozási alapelvével: nagy geometriai különbség kis átmeneti súlyt, kis különbség nagy átmeneti súlyt eredményez.

A seed lapok rögzített peremfeltételek. Legyen P_i^l annak valószínűsége, hogy az f_i lapból induló véletlen séta először az l -edik seedet éri el. A nem seed lapokra a harmonikus egyenlet:

$$\sum_{j \in N(i)} w_{ij} (P_i^l - P_j^l) = 0, \quad (3.4)$$

ahol $N(i)$ az f_i lap szomszédainak halmaza. Ebből a program egy ritka Laplace-mátrixot épít. Ez az egyenlet közvetlenül megfelel a *Rapid and effective segmentation of 3D models using random walks* random walk szegmentálásánál használt harmonikus egyenletének. A seed szomszédságok ismert értékei a jobb oldalra kerülnek. A mátrix ritka, ezért az implementáció Eigen `SimplicialLDLT` faktorizációt használ. A megoldás után minden lap ahhoz a seedhez kap címkét, amelyhez a legnagyobb valószínűség tartozik:

$$\text{label}(f_i) = \arg \max_l P_i^l. \quad (3.5)$$

Ez a címkézési szabály szintén megegyezik Lai és munkatársai döntési lépésével.

3.3.2. Automatikus seedválasztás

Az automatikus seedválasztás célja, hogy a felhasználónak ne kelljen kézzel megadnia a kiinduló lapokat. A durva módban a program kevés, egymástól távol lévő seedet keres. Ehhez először minden összefüggő komponens kap legalább egy seedet, majd a további seedek farthest-point stratégia szerint kerülnek kiválasztásra. Ha S az eddig kiválasztott seedek halmaza, akkor egy lap aktuális távolsága:

$$D(f_i, S) = \min_{s \in S} \text{dist}(f_i, s), \quad (3.6)$$

a következő seed pedig:

$$s_{\text{next}} = \arg \max_{f_i} D(f_i, S). \quad (3.7)$$

A `selectAutomaticSeedsCoarse()` ezt a távolságot Dijkstra-algoritmussal számolja a lapgráfon. Ez a seedeket a modell felületén elosztva helyezi el, így alkalmas nagyobb szerkezeti részek elkülönítésének inicializálására. A farthest-point stratégia Lai és munkatársai durva seedválasztási gondolatával egyezik meg, de a projekt egyszerűbb távolságdefiníciót használ.

A finom mód más elvet követ. Itt a cél nem azonnal a végleges régiószám előállítása, hanem egy sűrűbb, jellemzőérzékeny túlszegmentálás. A program minden laphoz fontossági értéket rendel, amely figyelembe veszi a lokális normálisváltozást, a konkávitást, a modell középpontjától mért relatív távolságot, mérnöki módban pedig a görbületi értékeket is. Ezt jelölje I_i . A kiválasztáskor a távolság és a fontosság együtt határozza meg a következő seedet: minél távolabb van egy lap a korábbi seedektől, és minél fontosabb lokális geometriai jellemzőkkel rendelkezik, annál nagyobb eséllyel lesz új seed. Ez nem azonos Lai és munkatársai feature-sensitive particle distribution eljárásával, hanem annak egyszerűbb, determinisztikus közelítése.

3.3.3. Régióösszevonás

A finom automatikus szegmentálás sok seedet használ, ezért a random walk megoldás után a modell gyakran túlszegmentált. A `mergeSegmentsByBoundaryCost()` feladata, hogy a gyenge határral elválasztott szomszédos régiókat összevonja. Két szomszédos szegmens legyen A és B . A közös határ súlyozott átlagos különbsége:

$$\bar{d}_{AB} = \frac{\sum_{e \in \partial A \cap \partial B} |e| d_e}{\sum_{e \in \partial A \cap \partial B} |e|}. \quad (3.8)$$

Ezt a program a két régió teljes külső határának átlagos különbségéhez hasonlítja:

$$\bar{d}_{A \cup B} = \frac{C_A + C_B - C_{AB}}{L_A + L_B - L_{AB}}, \quad (3.9)$$

ahol C a határon összegzett $|e|d_e$ értéket, L pedig a határhosszt jelöli. Az összevonási költség:

$$m(A, B) = \frac{\bar{d}_{AB}}{\bar{d}_{A \cup B}}. \quad (3.10)$$

Ez a költség Lai és munkatársai régióösszevonási képletének megfelelő szerepet tölti be: azt méri, hogy a két régió közös határa mennyire gyenge a régiók többi határához képest. Kis költség esetén az összevonás indokolt. A program mindig a legkisebb költségű szomszédos régiópárt vonja össze, majd újraszámolja a határstatisztikákat. A folyamat addig tart, amíg el nem éri a célként megadott régiószámot, vagy amíg a legjobb összevonási költség meghaladja a felhasználó által megadott határt. A finom mód után egy külön tisztítási lépés a nagyon kis háromszögszámú régiókat is képes a leggyengébb szomszédos határ mentén beolvasztani.

3.4. Megjelenítés és felhasználói interakció

A megjelenítési réteg feladata, hogy az algoritmus eredményét interaktívan ellenőrizhető formában mutassa meg. A felhasználó a kamerát forgathatja és nagyíthatja, modellt válthat, seed háromszögeket jelölhet ki, illetve automatikus szegmentálási módokat indíthat. A `main.cpp` tartja kézben az alkalmazás fő ciklusát: feldolgozza a bemenetet, frissíti az állapotot, szükség esetén újraszámolja a szegmentációt, majd átadja a renderelőnek a kirajzolandó címkéket és vezérlési paramétereket.

3.4.1. Renderelés bgfx-szel

A `Renderer` osztály a CPU oldali hálóból GPU oldali buffereket készít. Az alapmodellhez egy vertexbuffer és egy indexbuffer tartozik. Szegmentációs előnézet esetén a program a háromszögeket címke szerint csoportosítja, és minden szegmenshez külön indexbuffert hoz létre. A seed háromszögek és a szegmenshatárok szintén külön indexbufferen keresztül rajzolódni ki, így az alapmodell, a színezett régiók, a fekete határvonalak és a piros seed kiemelések egymástól független rajzolású lépések maradnak.

A kamera orbit jellegű: a felhasználó a modell körül forgatja a nézőpontot, a modell pedig az origó körül helyezkedik el. A vertex shader a szokásos modell-, nézeti és vetítési transzformációkat végzi el, a fragment shader pedig egyszerű ambient, diffúz és spekuláris komponensekből álló megvilágítást számol. Ez a rész nem kapcsolódik közvetlenül Lai és munkatársai szegmentálási képleteihez; a célja csupán az, hogy a modell formája és a szegmentációs eredmény jól látható legyen.

A szegmentációs előnézetnél az alapszín nem egységes, hanem a háromszög címkéjéből származik. Az azonos címkéjű háromszögek azonos palettaszínt kapnak, a különböző címkéjű szomszédos lapok közös élei pedig fekete vonalként jelennek meg. Ez közvetlen vizuális visszajelzést ad arról, hol húzódnak a számított szegmenshatárok.

3.4.2. Felhasználói felület

A felhasználói felület ImGui alapú. A `RendererUiState` egyetlen állapotstruktúrában tárolja a kamera, a világítás, a szegmentációs mód, az automatikus seedelés és a régióösszevonás paramétereit. A felület két fő panelt használ: az egyik a renderelési és modellválasztási beállításokat, a másik a szegmentációs vezérlőket tartalmazza.

Kézi módban a felhasználó seed háromszögeket jelölhet ki. A kijelöléshez a program a kurzor pozíciójából sugarat képez, majd CPU oldalon metszi ezt a modell háromszögeivel. A legközelebbi metszett háromszög lesz az új seed. Automatikus módban a kézi seedelés le van tiltva, és a felület a durva vagy finom seedválasztáshoz tartozó paramétereket módosítja.

Az UI közvetlenül nem futtatja az algoritmust, hanem csak állapotváltozásokat és kéréseket ad vissza a fő ciklusnak. Ilyen kérés például a modellváltás, a seedek törlése, véletlen seed elhelyezése, automatikus szegmentálási lépés indítása vagy a teljes automatikus folyamat befejezése. Ez a szétválasztás azért hasznos, mert a felület kezelése, az algoritmus futtatása és a renderelés egymástól elkülönített felelősségi kör marad.

3.5. Implementációs különbségek

A SegMesh nem Lai és munkatársai *Rapid and effective segmentation of 3D models using random walks* című munkájának teljes, minden részletre kiterjedő újrainplementációja, hanem annak egy interaktív, háromszöghálókra fókuszáló prototípusa. A legfontosabb egyezés az interaktív random walk szegmentálás magjában található: a program a háromszöglapokból gráfot épít, a seed lapokat rögzített címkékként kezeli, majd egy ritka lineáris egyenletrendszer megoldásával rendeli hozzá a nem seed lapokat a legvalószínűbb régióhoz. Ez a rész követi legközvetlenebbül a Lai-féle módszer alap gondolatát.

Az egyik lényeges eltérés, hogy Lai és munkatársai a háromszöghálók mellett pontfelhők szegmentálását is tárgyalják, a SegMesh viszont csak .obj formátumú háromszöghálókkal dolgozik. Emiatt a bemenet feldolgozása OpenMesh alapú hálóbetalöltésre, laptulajdonságok számítására és lapszomszédosági gráf felépítésére épül. Pontfelhőkhöz tartozó szomszédoságépítés vagy külön pontfelhő alapú random walk gráf nem készül.

Az automatikus szegmentálásban szintén vannak egyszerűsítések. A durva mód a Lai-féle távoli seedválasztási gondolatot követi, de a projekt elsősorban centroidok közötti, geodetikus jellegű távolságokra támaszkodó farthest-point stratégiát használ. A finom mód csak közelítése az eredeti módszernek: Lai és munkatársai feature-sensitive particle distribution eljárása helyett a program determinisztikus fontossági értéket és ismételt Dijkstra-számítást használ a sűrűbb seedeléshez.

Lai és munkatársai munkájában szereplő hierarchikus szegmentálás és a végső határsimítás nem része a jelenlegi implementációnak. A SegMesh egyetlen, lapos szegmentációs szintet jelenít meg, vagyis nem épít rekurzív régiófát, és nem kínál több részletességi szint között váltható hierarchiát. A megjelenített határok a háromszöglapok címkéiből közvetlenül adódnak, ezért a szegmenshatárok a háló felbontását követik, külön simítás vagy határkorrekció nélkül.

Az engineering mód is egyszerűsített. Az eredeti módszer robusztusabb görbületi és felületi jellemzőket használ a mérnöki modellek kezeléséhez, míg a projektben a

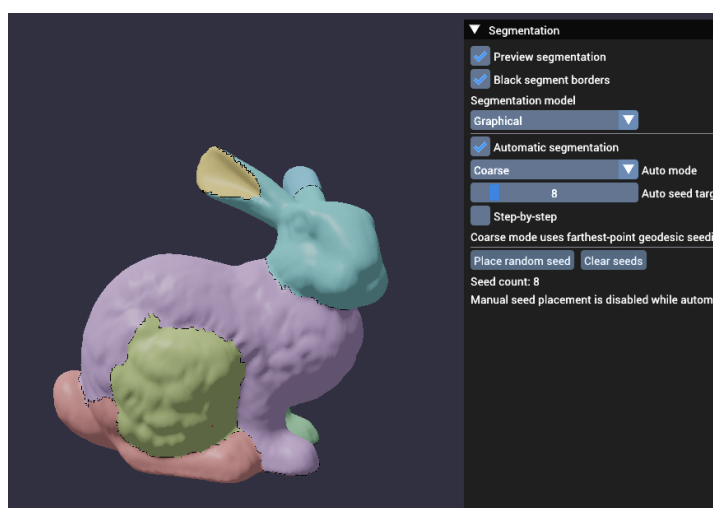
normáliskülönbség mellett becsült Gauss- és középgörbületi különbségek szerepelnek. Ez gyakorlatban használható heurisztikát ad, de nem azonos a Lai és munkatársai által leírt integrális invariáns alapú görbületbecsléssel.

További különbség, hogy az eredeti munka több automatikus megállási feltételt is tárgyal, például a seedek számának vagy az összevonás leállításának meghatározására. A SegMesh ehelyett több helyen felhasználói paraméterekre támaszkodik: a seedek célzott száma, a finom szegmensek összevonási határa és a végső régiószám az ImGui felületen állítható. Ez kevésbé automatikus, viszont jobban illik egy kísérleti alkalmazáshoz, mert a különböző beállítások hatása azonnal kipróbálható és vizuálisan ellenőrizhető.

4. fejezet

Eredmények

A tesztelés alapján a kézi seedeléssel indított *graphical* és *engineering* szegmentálási módok alapvetően jól működnek. A *graphical* beállítás elsősorban a szomszédos lapok normálisainak eltérésére és a konkáv-konvex élek kezelésére támaszkodik, ezért jól használható olyan modelleken, ahol a természetes szegmenshatárok erősebb geometriai töréseknél jelennek meg. Az eredményen jól látható, hogy a módszer a modell nagyobb, vizuálisan elkülönülő részeit képes szétválasztani.

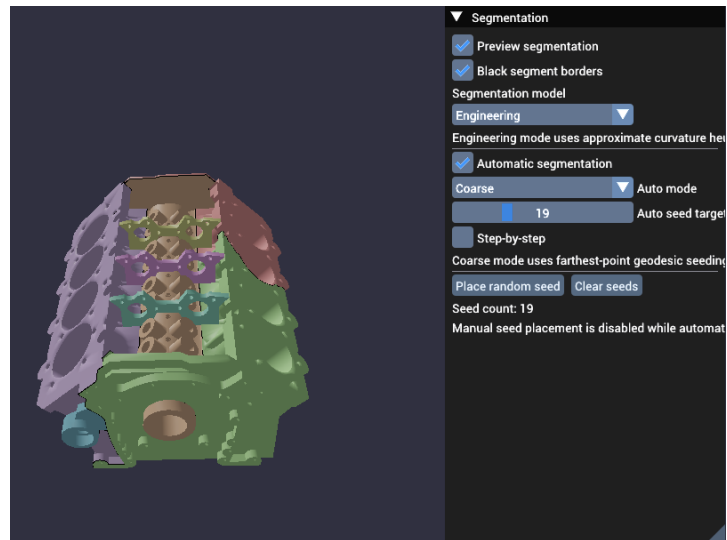


4.1. ábra. Graphical szegmentálási eredmény.

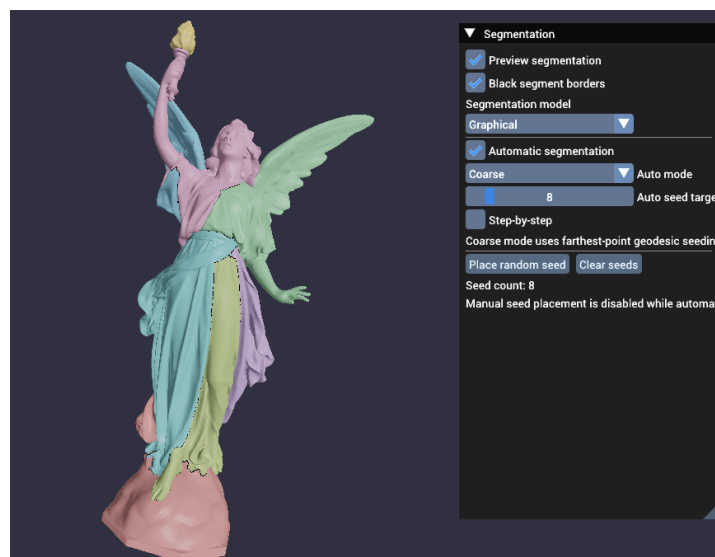
Az *engineering* mód a normáliskülönbség mellett görbületi jellemzőket is figyelembe vesz. Emiatt jobban illeszkedik azokhoz a mérnöki modellekhez, amelyeknél nemcsak a vizuálisan feltűnő törések, hanem a különböző felülettípusok elválasztása is fontos. A módszer a vizsgált példán értelmezhető, jól elkülönülő régiókat ad, így a mérnöki jellegű szegmentálási célhoz használható kiindulási alapot biztosít.

A durva automatikus szegmentálás szintén alapvetően jól működik: a program kézi kijelölés nélkül is képes kezdő seedeket választani, majd ezekből értelmezhető régiókat létrehozni. A seedek elosztásánál azonban modelltől függően problémát okozhat, hogy az algoritmus a legtávolabbi pontokba helyezi az új seedeket. Ilyenkor a számítás a modell legextrémabb pontjaihoz próbál alkalmazkodni, ami bizonyos alakzatoknál kevésbé természetes régiófelosztást eredményezhet.

A finom automatikus szegmentálás jelenlegi implementációja erősen függ a modelltől és a választott beállításoktól. A sűrűbb seedelés és az azt követő régióösszevonás képes részletesebb felosztást létrehozni, de az eredmény minősége érzékeny arra, hogy a seedek

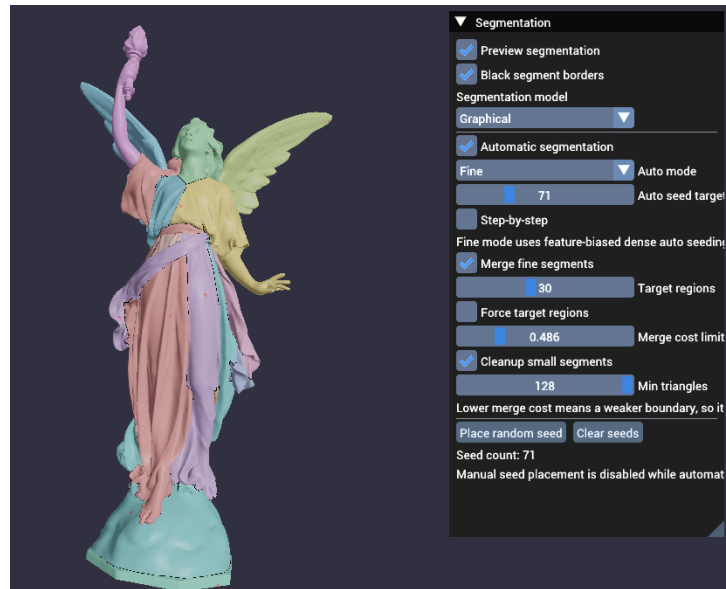


4.2. ábra. Engineering szegmentálási eredmény.



4.3. ábra. Durva automatikus szegmentálási eredmény.

hogyan oszlanak el a felületen. Emiatt a seed elosztására érdemes lenne más módszert is alkalmazni, illetve az összefésülési algoritmuson is lehetne javítani. A jelenlegi változat így inkább kísérleti jellegű: jól szemlélteti a finomabb automatikus szegmentálás lehetőségét, de további fejlesztést igényel ahhoz, hogy stabilan jó eredményt adjon különböző modelleken.



4.4. ábra. Finom automatikus szegmentálási eredmény.

5. fejezet

Összefoglalás

Az önálló laboratóriumi munka során egy olyan interaktív alkalmazás készült, amely háromdimenziós háromszöghálók szegmentálását támogatja. A projekt kiindulópontját a random walk alapú szegmentálási módszer adta, amely a modellt gráfként kezeli, és a felhasználó által kijelölt vagy automatikusan választott seedek alapján rendel címkéket a háló lapjaihoz. A megvalósítás manuális, durva automatikus és finom automatikus munkafolyamatot is tartalmaz, így a szegmentálási eredmények többféle beállítással vizsgálhatók.

A fejlesztés során elkészült a modellek betöltéséhez és előfeldolgozásához szükséges adatmodell, a random walk megoldó, az automatikus seedválasztás és a régióösszevonás alapvető logikája. A megjelenítés a **bgfx** könyvtárra épül, a felhasználói felület pedig közvetlenül láthatóvá teszi a seedeket, a szegmenshatárokat és a különböző algoritmikus beállítások hatását. A projekt nem teljes CAD-rekonstrukciós rendszer, hanem egy olyan prototípus, amely a szegmentálási lépés működését és gyakorlati használhatóságát helyezi előtérbe.

5.1. Jövőbeli tervek

A projekt egyik fontos továbbfejlesztési lehetősége a határsimítás implementálása. Lai és munkatársai random walk alapú szegmentálási módszerében a szegmentálás után egy post processing lépés is szerepel, amely a nyers szegmenshatárokat javítja és simítja [1]. Ez a jelenlegi programból még hiányzik: a megjelenített határok közvetlenül a háromszöglapok címkéiből adódnak, ezért erősen követik a háló lokális felbontását. A határsimítás implementálása természetesebb, vizuálisan tisztább szegmenshatárokat eredményezhetne.

További fejlesztési irány az automatikus seedelosztás módosítása. Az eredményeknél látható volt, hogy a jelenlegi durva automatikus módszer bizonyos modelleken túl erősen a legtávolabbi, szélső pontokhoz igazodik. Emiatt érdemes lenne új seedválasztási stratégiát hozzáadni, vagy a meglévő farthest-point eljárást kiegészíteni olyan heurisztikákkal, amelyek jobban figyelembe veszik a modell lokális geometriáját és a várható szegmenshatárokat.

Egyszerűbb modelleken a durva szegmentálás egy valós idejű rombolási rendszer alapja is lehetne. Ebben az esetben a program a szegmenseket külön objektumokra bontaná, majd az így létrejövő nyitott oldalakat kitöltené. Ez lehetővé tenné, hogy egy modell előre számított vagy gyorsan előállítható szegmensekből essen szét, miközben a darabok önálló objektumként tovább kezelhetők lennének egy fizikai vagy grafikai alkalmazásban.

6. fejezet

Hivatkozások

- [1] Yu-Kun Lai, Shi-Min Hu, Ralph R. Martin és Paul L. Rosin. *Rapid and effective segmentation of 3D models using random walks*. Computer Aided Geometric Design, 26:665–679, 2009.
- [2] Panagiotis Theologou, Ioannis Pratikakis és Theoharis Theoharis. *A comprehensive overview of methodologies and performance evaluation frameworks in 3D mesh segmentation*. Computer Vision and Image Understanding, 135:49–82, 2015.
- [3] Tamás Várady, Pál Benkő és Géza Kós. *Reverse engineering regular objects: simple segmentation and surface fitting procedures*. International Journal of Shape Modeling, 4:127–141, 1998.
- [4] Tamás Várady, Michael A. Facello és Zsolt Terék. *Automatic extraction of surface structures in digital shape reconstruction*. Computer-Aided Design, 39:379–388, 2007.
- [5] Pierre-Alain Fayolle és Alexander Pasko. *Segmentation of discrete point clouds using an extensible set of templates*. The Visual Computer, 29:449–465, 2013.
- [6] Shapr3D. *CSG Reconstruction of 3D Scans*. Technical report, 2020.
- [7] Zygle. *SegMesh*. <https://github.com/Zygle/SegMesh>.